

- Description

This module manages a smart electric vehicle regenerative braking system, ensuring that:

1. The system continuously monitors the vehicle's current speed (in km/h) and the deceleration rate (in km/h per second) during braking.
2. If the deceleration rate exceeds a predefined threshold, regenerative braking is activated to recover energy.
3. The recovered energy (in kWh) is calculated using a numerical formula based on the deceleration rate and current speed.
4. If the deceleration rate is below the threshold, no energy recovery occurs.

This system helps improve overall energy efficiency and extends the vehicle's battery life by recovering braking energy.

- Specification

module SmartEVRegenerativeBrakingControl

var currentSpeed, decelerationRate, recoveredEnergy, regenThreshold, efficiencyFactor: real

process updateDeceleration(newDeceleration: real) returns (recoveredEnergy: real, currentSpeed: real)

functional scenario form:

decelerationRate >= regenThreshold && recoveredEnergy = currentSpeed * decelerationRate * efficiencyFactor, output "Deceleration: " + decelerationRate + " km/h/s exceeds threshold " + regenThreshold + " km/h/s. Recovered Energy: " + recoveredEnergy + " kWh.", Regenerative braking activated and energy recovered calculated.

||

decelerationRate < regenThreshold && recoveredEnergy = 0, output "Deceleration: " + decelerationRate + " km/h/s below threshold " + regenThreshold + " km/h/s. No energy recovery.", No regenerative braking; energy recovery is 0.

end process

end module

- Java Code

```
package code;
public class c89_SmartEVRegenerativeBraking {
    private double currentSpeed;
    private double decelerationRate;
    private double recoveredEnergy;
    private double regenThreshold;
    private double efficiencyFactor;
    public c89_SmartEVRegenerativeBraking(double initialSpeed, double r
egenThreshold, double efficiencyFactor) {
        this.currentSpeed = initialSpeed;
        this.decelerationRate = 0.0;
```

```

        this.recoveredEnergy = 0.0;
        this.regenThreshold = regenThreshold;
        this.efficiencyFactor = efficiencyFactor;
    }
    public void updateDeceleration(double newDecelerationRate) {
        decelerationRate = newDecelerationRate;
        if (decelerationRate >= regenThreshold) {
            recoveredEnergy = currentSpeed * decelerationRate * efficiencyFactor;
            System.out.printf("Deceleration Rate: %.2f km/h/s exceeds threshold %.2f km/h/s.%n", decelerationRate, regenThreshold);
            System.out.printf("Recovered Energy: %.4f kWh%n", recoveredEnergy);
            System.out.println("Regenerative braking activated.");
        } else {
            recoveredEnergy = 0.0;
            System.out.printf("Deceleration Rate: %.2f km/h/s below threshold %.2f km/h/s.%n", decelerationRate, regenThreshold);
            System.out.println("No regenerative braking.");
        }
    }
    public void updateSpeed(double newSpeed) {
        currentSpeed = newSpeed;
        System.out.printf("Current Speed updated to: %.2f km/h%n", currentSpeed);
    }
    public double getRecoveredEnergy() {
        return recoveredEnergy;
    }
    public double getCurrentSpeed() {
        return currentSpeed;
    }
}

public static void main(String[] args) {
    c89_SmartEVRegenerativeBraking evRegen = new c89_SmartEVRegenerativeBraking(80.0, 10.0, 0.0005);

    // testcase-VT:
    // evRegen.updateSpeed(80.0);
    // evRegen.updateDeceleration(8.0);
    // evRegen.updateDeceleration(12.0);
    // evRegen.updateSpeed(60.0);
    // evRegen.updateDeceleration(15.0);

```

```

// testcase-FT:
//     evRegen.updateDeceleration(2.1537143965193995E291);

//     evRegen.updateSpeed(-1.7976931348623157E308);
//     evRegen.updateSpeed(2.3238675791584434E29);
//     evRegen.updateSpeed(7.402265849433064E307);
//     evRegen.updateDeceleration(5.992310449541052E307);
// testcase-Z3:
//     evRegen.updateSpeed(80.0);
//     evRegen.updateDeceleration(8.0);
//     evRegen.updateDeceleration(12.0);
//     evRegen.updateSpeed(60.0);
//     evRegen.updateDeceleration(15.0);

// testcase-UVT:
evRegen.updateSpeed(80.0);
evRegen.updateDeceleration(8.0);
evRegen.updateDeceleration(12.0);
evRegen.updateSpeed(60.0);
evRegen.updateDeceleration(15.0);
}
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c89_SmartEVRegenerativeBraking.java	100.0 %	2	0	2
▼ c89_SmartEVRegenerativeBraking	100.0 %	2	0	2
main(String[])		0	0	0
c89_SmartEVRegenerativeBraking		0	0	0
updateDeceleration(double)	100.0 %	2	0	2
updateSpeed(double)		0	0	0

3. Vibration testing instruction coverage: 95.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	135	21,457	21,592
▼ src	0.6 %	135	21,457	21,592
▼ code	0.6 %	135	21,457	21,592
▼ c89_SmartEVRegenerativeBraking.java	95.7 %	135	6	141
▼ c89_SmartEVRegenerativeBraking	95.7 %	135	6	141
main(String[])	100.0 %	23	0	23
c89_SmartEVRegenerativeBraking	100.0 %	18	0	18
updateDeceleration(double)	100.0 %	78	0	78
updateSpeed(double)	100.0 %	16	0	16

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.1 %	1	841	842
▼ src	0.1 %	1	841	842
▼ code	0.1 %	1	841	842
▼ c89_SmartEVRegenerativeBraking.java	50.0 %	1	1	2
▼ c89_SmartEVRegenerativeBraking	50.0 %	1	1	2
main(String[])		0	0	0
c89_SmartEVRegenerativeBraking		0	0	0
updateDeceleration(double)	50.0 %	1	1	2
updateSpeed(double)		0	0	0

5. Fuzz testing instruction coverage: 78.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	111	21,481	21,592
▼ src	0.5 %	111	21,481	21,592
▼ code	0.5 %	111	21,481	21,592
▼ c89_SmartEVRegenerativeBraking.java	78.7 %	111	30	141
▼ c89_SmartEVRegenerativeBraking	78.7 %	111	30	141
main(String[])	100.0 %	23	0	23
c89_SmartEVRegenerativeBraking	100.0 %	18	0	18
updateDeceleration(double)	69.2 %	54	24	78
updateSpeed(double)	100.0 %	16	0	16

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c89_SmartEVRegenerativeBraking.java	100.0 %	2	0	2
▼ c89_SmartEVRegenerativeBraking	100.0 %	2	0	2
main(String[])		0	0	0
c89_SmartEVRegenerativeBraking		0	0	0
updateDeceleration(double)	100.0 %	2	0	2
updateSpeed(double)		0	0	0

7. Z3 instruction coverage: 95.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	135	22,894	23,029
▼ src	0.6 %	135	22,894	23,029
▼ code	0.6 %	135	22,894	23,029
▼ c89_SmartEVRegenerativeBraking.java	95.7 %	135	6	141
▼ c89_SmartEVRegenerativeBraking	95.7 %	135	6	141
main(String[])	100.0 %	23	0	23
c89_SmartEVRegenerativeBraking	100.0 %	18	0	18
updateDeceleration(double)	100.0 %	78	0	78
updateSpeed(double)	100.0 %	16	0	16

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c89_SmartEVRegenerativeBraking.java	100.0 %	2	0	2
▼ c89_SmartEVRegenerativeBraking	100.0 %	2	0	2
main(String[])		0	0	0
c89_SmartEVRegenerativeBraking		0	0	0
updateDeceleration(double)	100.0 %	2	0	2
updateSpeed(double)		0	0	0

9. Unimproved V-method instruction coverage: 95.7%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  VibrationTestingTool		 0.6 %	135	22,894	23,029
▼  src		 0.6 %	135	22,894	23,029
▼  code		 0.6 %	135	22,894	23,029
▼  c89_SmartEVRegenerativeBraking.java		 95.7 %	135	6	141
▼  c89_SmartEVRegenerativeBraking		 95.7 %	135	6	141
 main(String[])		 100.0 %	23	0	23
 c89_SmartEVRegenerativeBraking(double)		 100.0 %	18	0	18
 updateDeceleration(double)		 100.0 %	78	0	78
 updateSpeed(double)		 100.0 %	16	0	16